

Python Assignment 2

Data Structures - List, Dictionary, Set -Conditional Statements

NAME:UMAYAL.K

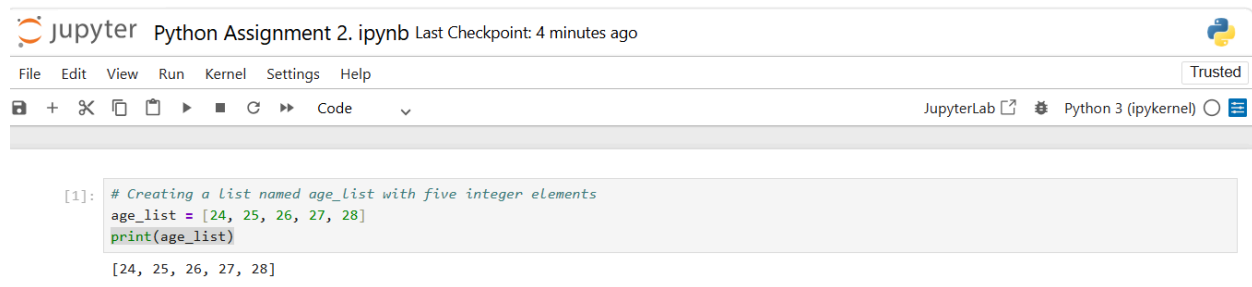
SUBMISSION DATE: 03-10-2026

PREVIEWS:

List (Creation, Modification and Access):

1. List Creation:

a.Create a list named `age_list` with five integer elements



The image shows a JupyterLab interface with a single code cell. The code cell contains a comment, a list creation statement, and a print statement. The output of the code cell is displayed below the code.

```
[1]: # Creating a list named age_list with five integer elements
age_list = [24, 25, 26, 27, 28]
print(age_list)
```

[24, 25, 26, 27, 28]

b. Create a list named `name_list` with five string elements.



The image shows a JupyterLab interface with two code cells. The first code cell contains a comment, a list creation statement, and a print statement. The output of the first code cell is displayed below the code. The second code cell contains a comment, a list creation statement, and a print statement. The output of the second code cell is displayed below the code.

```
[1]: # Creating a list named age_list with five integer elements
age_list = [24, 25, 26, 27, 28]
print(age_list)
```

[24, 25, 26, 27, 28]

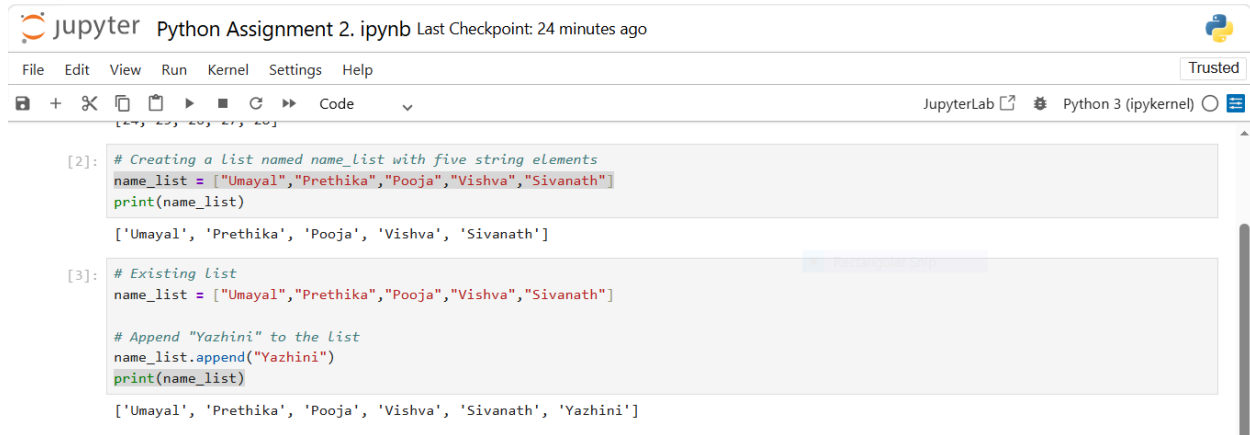
```
[2]: # Creating a list named name_list with five string elements
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath"]
print(name_list)
```

['Umayal', 'Prethika', 'Pooja', 'Vishva', 'Sivanath']

2. List Operations / Modifications:

a. Append the string "Yazhini" to name_list

- Used the `.append()` method in Python
- This adds "Yazhini" as the **last element** of the list.



The screenshot shows a JupyterLab window titled "Python Assignment 2. ipynb" with a "Trusted" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor displays two code cells. Cell [2] creates a list named 'name_list' with five string elements: 'Umayal', 'Prethika', 'Pooja', 'Vishva', and 'Sivanath', and prints it. Cell [3] shows the existing list, appends the string 'Yazhini' to it using the `.append()` method, and prints the updated list. The output of cell [3] shows the list with 'Yazhini' added at the end.

```
[2]: # Creating a list named name_list with five string elements
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath"]
print(name_list)

['Umayal', 'Prethika', 'Pooja', 'Vishva', 'Sivanath']

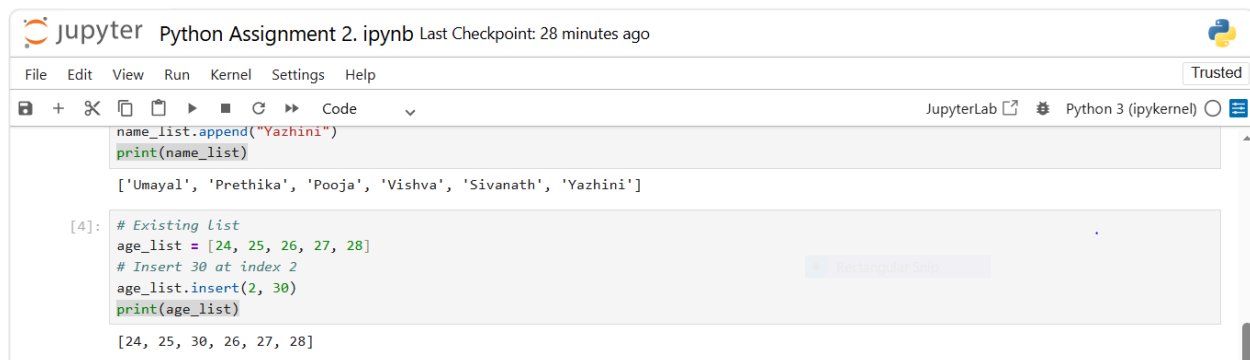
[3]: # Existing List
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath"]

# Append "Yazhini" to the List
name_list.append("Yazhini")
print(name_list)

['Umayal', 'Prethika', 'Pooja', 'Vishva', 'Sivanath', 'Yazhini']
```

b. Insert the element 30 at index 2 in age_list.

- This places 30 at the **third position** (since indexing starts from 0).



The screenshot shows a JupyterLab window titled "Python Assignment 2. ipynb" with a "Trusted" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The code editor displays two code cells. Cell [3] appends 'Yazhini' to 'name_list' and prints it. Cell [4] creates a list named 'age_list' with the values [24, 25, 26, 27, 28], inserts the value 30 at index 2 using the `.insert()` method, and prints the updated list. The output of cell [4] shows the list [24, 25, 30, 26, 27, 28].

```
name_list.append("Yazhini")
print(name_list)

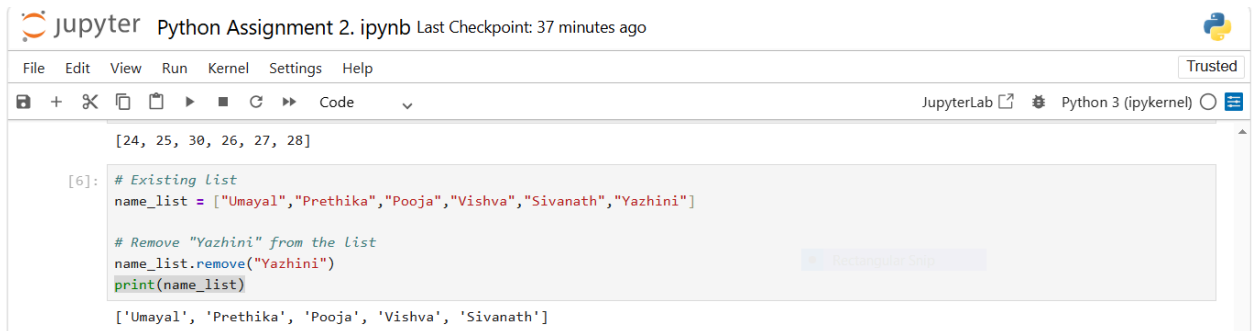
['Umayal', 'Prethika', 'Pooja', 'Vishva', 'Sivanath', 'Yazhini']

[4]: # Existing List
age_list = [24, 25, 26, 27, 28]
# Insert 30 at index 2
age_list.insert(2, 30)
print(age_list)

[24, 25, 30, 26, 27, 28]
```

c. Remove the string "Yazhini" from name_list.

- Used the `.remove()` method in Python
- This deletes the first occurrence of "Yazhini" from the list.



The screenshot shows a JupyterLab window titled "Python Assignment 2. ipynb" with a "Trusted" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor displays the following Python code:

```
[24, 25, 30, 26, 27, 28]

[6]: # Existing List
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath", "Yazhini"]

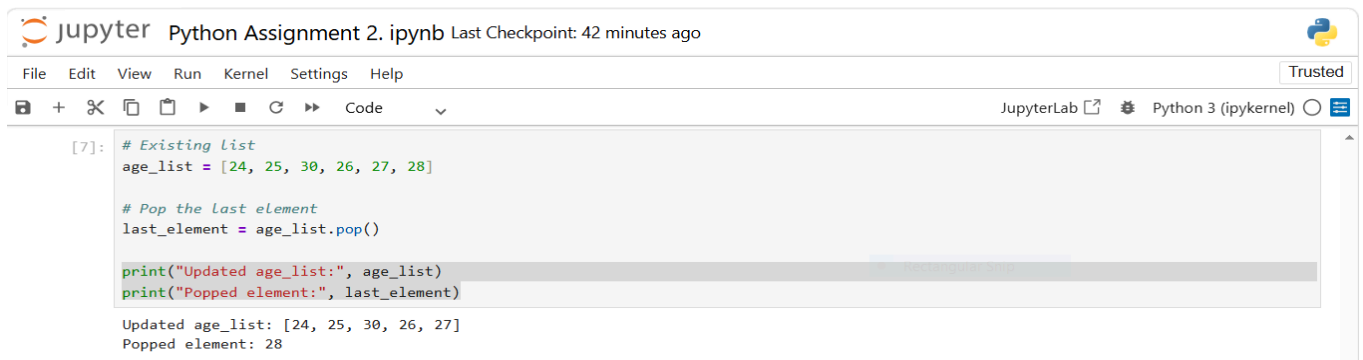
# Remove "Yazhini" from the List
name_list.remove("Yazhini")
print(name_list)
```

The output of the code is displayed below the editor:

```
['Umayal', 'Prethika', 'Pooja', 'Vishva', 'Sivanath']
```

d. Pop the last element from age_list.

- The `.pop()` method removes the last item by default and returns it, so you can store or display it separately.



The screenshot shows a JupyterLab window titled "Python Assignment 2. ipynb" with a "Trusted" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor displays the following Python code:

```
[7]: # Existing List
age_list = [24, 25, 30, 26, 27, 28]

# Pop the last element
last_element = age_list.pop()

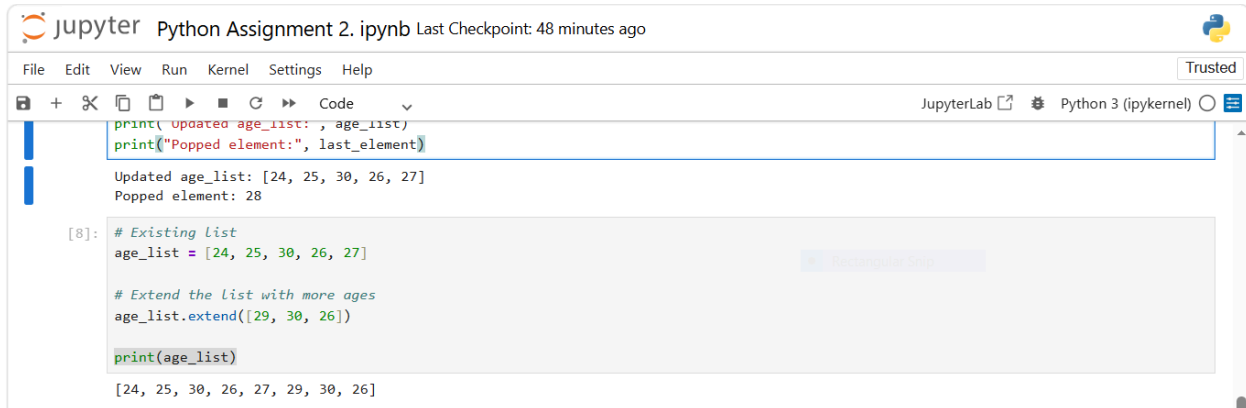
print("Updated age_list:", age_list)
print("Popped element:", last_element)
```

The output of the code is displayed below the editor:

```
Updated age_list: [24, 25, 30, 26, 27]
Popped element: 28
```

e. Extend the `age_list` with additional ages [29, 30, 26].

- The `.extend()` method adds multiple elements from another list to the end of the existing list



The screenshot shows a JupyterLab window titled "Python Assignment 2. ipynb" with a "Trusted" status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The code cell contains the following Python code:

```
print(updated_age_list, age_list)
print("Popped element:", last_element)

Updated age_list: [24, 25, 30, 26, 27]
Popped element: 28

[8]: # Existing List
age_list = [24, 25, 30, 26, 27]

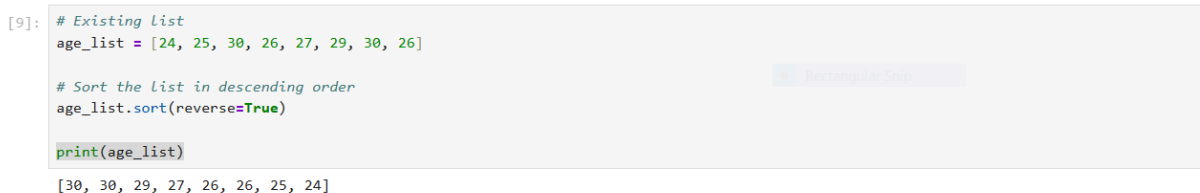
# Extend the List with more ages
age_list.extend([29, 30, 26])

print(age_list)

[24, 25, 30, 26, 27, 29, 30, 26]
```

f. Sort `age_list` in descending order.

- The `reverse=True` argument tells Python to sort the list from **highest to lowest**.



The screenshot shows a JupyterLab window with a code cell containing the following Python code:

```
[9]: # Existing List
age_list = [24, 25, 30, 26, 27, 29, 30, 26]

# Sort the list in descending order
age_list.sort(reverse=True)

print(age_list)

[30, 30, 29, 27, 26, 26, 25, 24]
```

g. Find Max age, Min age and sum of all ages from `age_list`.

- Used Python's built-in functions
- `max()` → finds the largest value
- `min()` → finds the smallest value
- `sum()` → adds up all elements in the list

```
[10]: # Existing list
age_list = [30, 30, 29, 27, 26, 26, 25, 24]

# Find max, min, and sum
max_age = max(age_list)
min_age = min(age_list)
total_age = sum(age_list)

print("Maximum age:", max_age)
print("Minimum age:", min_age)
print("Sum of all ages:", total_age)

Maximum age: 30
Minimum age: 24
Sum of all ages: 217
```

3. Accessing List Elements:

a. Print the first element of name_list

- To print the first element of `name_list`, used index `0` (since Python lists are zero-indexed)
- This retrieves the very first string in the list.

```
[11]: # Existing list
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath"]

# Print the first element
print(name_list[0])

Umayal
```

b. Print the last element of name_list.

- To print the last element of `name_list`, used negative indexing (`-1`)

```
[12]: # Existing list
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath"]

# Print the last element
print(name_list[-1])

Sivanath
```

c. Print the elements from index 2 to index 4 in name_list.

- To print the elements from index 2 to index 4 in `name_list`, used list slicing in Python
- The slice `[2:5]` starts at index `2` and goes up to but not including index `5`.
- So, it prints the elements at indices `2`, `3`, and `4`.

```
[13]: # Existing List
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath"]

# Print elements from index 2 to 4
print(name_list[2:5])

['Pooja', 'Vishva', 'Sivanath']
```

Activate Windows

d. Print the elements of `name_list` in reverse order.

- To print the elements of `name_list` in reverse order, used Python's slicing with a step of `-1`:
- The slice `[::-1]` tells Python to take the list from start to end but step backward, effectively reversing it.

```
[14]: # Existing List
name_list = ["Umayal", "Prethika", "Pooja", "Vishva", "Sivanath"]

# Print elements in reverse order
print(name_list[::-1])

['Sivanath', 'Vishva', 'Pooja', 'Prethika', 'Umayal']
```

4.Dictionary (Creation, Modification and Access):

a. Create a dictionary named `student_marks` that maps the names of five students to their marks (use scale of from 0 to 100).

- Creating a dictionary named **`student_marks`**
- A dictionary in Python stores data as key-value pairs.
- Here, each student's name is the key, and their mark is the value.

```
[17]: # Creating a dictionary named student_marks
student_marks = {"Umayal":95,"Prethika":92,"Pooja":76,"Vishva":88,"Sivanath":85}

print(student_marks)

{'Umayal': 95, 'Prethika': 92, 'Pooja': 76, 'Vishva': 88, 'Sivanath': 85}
```

b. Access and print the mark of a specific student, of your choice.

Used the student's name as the key

- In a dictionary, you access values using their **keys**.

- Here, "Prethika" is the key, and 92 is the value associated with it.

```
[18]: # Existing dictionary
student_marks = {"Umayal":95,"Prethika":92,"Pooja":76,"Vishva":88,"Sivanath":85}

# Access and print the mark of a specific student
print("Prethika's mark:", student_marks["Prethika"])

Prethika's mark: 92
```

c. Add a new student "Janani" with a mark of 80 to the student_marks dictionary.

- Added a new student "Janani" with a mark of 80 in student_marks dictionary
- You can add a new key-value pair to a dictionary simply by assigning a value to a new key.
- If the key already exists, this will update its value instead.

```
[19]: # Existing dictionary
student_marks = {"Umayal":95,"Prethika":92,"Pooja":76,"Vishva":88,"Sivanath":85}

# Add a new student "Janani" with mark 80
student_marks["Janani"] = 80

print(student_marks)

{'Umayal': 95, 'Prethika': 92, 'Pooja': 76, 'Vishva': 88, 'Sivanath': 85, 'Janani': 80}
```

d. Update the mark of any one older student to 82.

- You can modify a dictionary value by reassigning it using its key.
- Here, "Pooja"’s mark is updated from 76 to 82.

```
[21]: # Existing dictionary
student_marks = {"Umayal":95,"Prethika":92,"Pooja":76,"Vishva":88,"Sivanath":85,"Janani":80}

# Update the mark of one older student (for example, "Pooja") to 82
student_marks["Pooja"] = 82

print(student_marks)

{'Umayal': 95, 'Prethika': 92, 'Pooja': 82, 'Vishva': 88, 'Sivanath': 85, 'Janani': 80}
```

e. Use the `keys()`, `values()`, and `items()` methods to print all keys, values, and key-value pairs in the `student_marks` dictionary.

- `keys()` → returns all dictionary keys (student names).
- `values()` → returns all dictionary values (marks).
- `items()` → returns all key–value pairs as tuples.

```
[22]: # Existing dictionary
student_marks = {"Umayal":95,"Prethika":92,"Pooja":76,"Vishva":88,"Sivanath":85,"Janani":80}

# Print all keys
print("Keys:", student_marks.keys())

# Print all values
print("Values:", student_marks.values())

# Print all key-value pairs
print("Items:", student_marks.items())

Keys: dict_keys(['Umayal', 'Prethika', 'Pooja', 'Vishva', 'Sivanath', 'Janani'])
Values: dict_values([95, 92, 76, 88, 85, 80])
Items: dict_items([('Umayal', 95), ('Prethika', 92), ('Pooja', 76), ('Vishva', 88), ('Sivanath', 85), ('Janani', 80)])
```

5.Sets (Operations):

a. Create a set called `my_set` with following values:

`['a','e','i','o','u','a','a','i']`

Analyse the output and provide explanation for the same.

- A set in Python is an unordered collection of unique elements.
- When you pass the list `['a','e','i','o','u','a','a','i']` into `set()`, all duplicate values (`'a'` and `'i'` appear multiple times) are automatically removed.
- The final set contains only unique vowels: `{'i', 'a', 'u', 'o', 'e'}`
- Since sets are unordered, the elements may not appear in the same order as you inserted them.

✓ **Key takeaway: Sets are perfect when you want to eliminate duplicates and work only with unique values.**

```
[24]: # Creating a set with repeated values
my_set = set(['a','e','i','o','u','a','a','i'])

print(my_set)

{'i', 'a', 'u', 'o', 'e'}
```

b. Attempt to change the value of `my_set[4] = 's'`. If code throws an error, provide an explanation.

```
[25]: my_set = {'i', 'a', 'u', 'o', 'e'}
      my_set[4] = 's'
```

TypeError Traceback (most recent call last)
Cell In[25], line 2
 1 my_set = {'i', 'a', 'u', 'o', 'e'}
----> 2 my_set[4] = 's'
TypeError: 'set' object does not support item assignment

Explanation:

- Sets are unordered collections, meaning they don't have indices like lists or tuples.
- You cannot access or modify elements using indexing (e.g., `my_set[4]`), because the elements in a set don't have a fixed position.
- Sets are also mutable in the sense that you can add or remove elements, but not change existing ones directly by index.

c. Create two sets:

set1 with values: {1, 3, 5, 7, 9}

set2 with values: {2, 3, 5, 8, 10}

- Each set contains unique elements enclosed in curly braces `{}`.
- Sets are unordered, so the order of elements may vary when printed.

```
[26]: # Creating two sets
      set1 = {1, 3, 5, 7, 9}
      set2 = {2, 3, 5, 8, 10}

      print("Set 1:", set1)
      print("Set 2:", set2)
```

Set 1: {1, 3, 5, 7, 9}
Set 2: {2, 3, 5, 8, 10}

d. Compute and print the union and intersection of set1 and set2.

- Union (`set1.union(set2)`): Combines all unique elements from both sets.
- Intersection (`set1.intersection(set2)`): Returns only the elements that appear in both set

```
[27]: # Existing sets
set1 = {1, 3, 5, 7, 9}
set2 = {2, 3, 5, 8, 10}

# Compute union and intersection
union_set = set1.union(set2)
intersection_set = set1.intersection(set2)

print("Union of set1 and set2:", union_set)
print("Intersection of set1 and set2:", intersection_set)
```

Union of set1 and set2: {1, 2, 3, 5, 7, 8, 9, 10}
Intersection of set1 and set2: {3, 5}

6.Operators & Conditional Statements :

(IF, ELIF, ELSE)

Performance Category Program:

1. Prompt user for Input. Score range should be from 0 to 10 (both inclusive).

- if score < 0 or score > 10:
- Enter your score (0-10): 11
- Invalid score! Please enter a value between 0 and 10.

```
# Conditional statements to categorize performance
if score < 0 or score > 10:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score >= 9:
    print("Performance Category: Excellent")
elif score >= 7:
    print("Performance Category: Good")
elif score >= 5:
    print("Performance Category: Average")
elif score >= 3:
    print("Performance Category: Poor")
else:
    print("Performance Category: Very Poor")
```

Enter your score (0-10): 11
Invalid score! Please enter a value between 0 and 10.

The program first checks if the score is valid (between 0 and 10).

Then it uses `if`, `elif`, and `else` to classify the score into categories:

- 9–10 → Excellent
- 7–8 → Good
- 5–6 → Average

- 3–4 → Poor
- 0–2 → Very Poor

elif score >= 9

- Enter your score (0-10): **9**
- Performance Category: **Excellent**

```
[29]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Conditional statements to categorize performance
if score < 0 or score > 10:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score >= 9:
    print("Performance Category: Excellent")
elif score >= 7:
    print("Performance Category: Good")
elif score >= 5:
    print("Performance Category: Average")
elif score >= 3:
    print("Performance Category: Poor")
else:
    print("Performance Category: Very Poor")

Enter your score (0-10): 9
Performance Category: Excellent
```

elif score >= 7

- Enter your score (0-10): **7**
- Performance Category: **Good**

```
[30]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Conditional statements to categorize performance
if score < 0 or score > 10:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score >= 9:
    print("Performance Category: Excellent")
elif score >= 7:
    print("Performance Category: Good")
elif score >= 5:
    print("Performance Category: Average")
elif score >= 3:
    print("Performance Category: Poor")
else:
    print("Performance Category: Very Poor")

Enter your score (0-10): 7
Performance Category: Good
```

elif score >= 5

- Enter your score (0-10): **5**
- Performance Category: **Average**

```
[31]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Conditional statements to categorize performance
if score < 0 or score > 10:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score >= 9:
    print("Performance Category: Excellent")
elif score >= 7:
    print("Performance Category: Good")
elif score >= 5:
    print("Performance Category: Average")
elif score >= 3:
    print("Performance Category: Poor")
else:
    print("Performance Category: Very Poor")

Enter your score (0-10): 5
Performance Category: Average
```

Activate Windows

elif score >= 3

- Enter your score (0-10): **3**
- Performance Category: **Poor**

```
[32]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Conditional statements to categorize performance
if score < 0 or score > 10:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score >= 9:
    print("Performance Category: Excellent")
elif score >= 7:
    print("Performance Category: Good")
elif score >= 5:
    print("Performance Category: Average")
elif score >= 3:
    print("Performance Category: Poor")
else:
    print("Performance Category: Very Poor")

Enter your score (0-10): 3
Performance Category: Poor
```

Activate Windows

2. Find the performance category based on the input score using following criteria:

- a. Above Average: Score greater than 7
- b. Average: Score between 4 and 7(both inclusive)
- c. Below Average: Score lesser than 4

- **if score > 10 or score < 0**
- Enter your score (0-10): -1
- **Invalid score!** Please enter a value between 0 and 10

```
[35]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Determine performance category
if score > 10 or score < 0:
    print("Invalid score! Please enter a value between 0 and 10.")

Enter your score (0-10): -1
Invalid score! Please enter a value between 0 and 10.
```

a. Above Average: Score greater than 7

elif score > 7

- Enter your score (0-10): 8
- Performance Category: **Above Average**

```
[37]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Determine performance category
if score > 10 or score < 0:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score > 7:
    print("Performance Category: Above Average")

Enter your score (0-10): 8
Performance Category: Above Average
```

b. Average: Score between 4 and 7(both inclusive)

elif score >= 4 and score <= 7

- Enter your score (0-10): 6
- Performance Category: **Average**

```
[38]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Determine performance category
if score > 10 or score < 0:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score > 7:
    print("Performance Category: Above Average")
elif score >= 4 and score <= 7:
    print("Performance Category: Average")

Enter your score (0-10): 6
Performance Category: Average
```

c. Below Average: Score lesser than 4

else:# score less than 4

- Enter your score (0-10): 3
- Performance Category: **Below Average**

```
[39]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Determine performance category
if score > 10 or score < 0:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score > 7:
    print("Performance Category: Above Average")
elif score >= 4 and score <= 7:
    print("Performance Category: Average")
else:# score less than 4
    print("Performance Category: Below Average")

Enter your score (0-10): 3
Performance Category: Below Average
```

3. Output: Print the Performance category

elif score >= 4 and score <= 7

- Enter your score (0-10): 5
- Performance Category: **Average**
- The program reads the user's score.
- It checks the range and prints the Performance Category accordingly.
- The output line `print("Performance Category: ...")` displays the result directly to the user.

```
[47]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Determine performance category
if score > 10 or score < 0:
    print("Invalid score! Please enter a value between 0 and 10.")
elif score > 7:
    print("Performance Category: Above Average")
elif score >= 4 and score <= 7:
    print("Performance Category: Average")
else:
    print("Performance Category: Below Average")
```

Enter your score (0-10): 5
Performance Category: Average

4. Additional Step: You can give a prompt of your choice to each category.
For eg: If score below average “Need to Improve your performance, consistent practice will lead to better results”.

else: # score less than 4

- Enter your score (0-10): 3
- Performance Category: **Below Average**
- **Need to improve your performance — consistent practice will lead to better results.**

```
[51]: # Prompt user for input
score = int(input("Enter your score (0-10): "))

# Determine performance category
if score > 10 or score < 0:
    print("Invalid score! Please enter a value between 0 and 10.")
else: # score less than 4
    print("Performance Category: Below Average")
    print("Need to improve your performance - consistent practice will lead to better results.")
```

Enter your score (0-10): 3
Performance Category: Below Average
Need to improve your performance - consistent practice will lead to better results.

